

Bootcamp: Full Stack Junior

Modulo 2: Ingeniería de Software y Análisis del Negocio

Semana 5

Contenido

1

Principios SOLID

2

Clean Code

3

Gestión ágil con Scrum

4

Gestión ágil con Kanban

Contenido:

1. Excelencia Técnica: Clean Code y Principios SOLID

Para desarrollar software profesional, no basta con que el código "funcione"; debe ser mantenible y escalable.

1.1. Clean Code (Código Limpio)

El código limpio es aquel que se puede leer y entender como si fuera una prosa bien escrita.

- **Nombres significativos:** Las variables y funciones deben revelar su intención (ej. `obtenerUsuarioActivo()` en lugar de `getU()`).
- **Funciones pequeñas:** Una función debe hacer solo una cosa y hacerla bien.
- **Comentarios innecesarios:** El código debe ser tan claro que no necesite explicaciones extra; usa comentarios solo para advertencias o decisiones de diseño críticas.
- **Menos comentarios, más claridad:** se prefieren nombres y estructuras claras antes que comentarios que expliquen código confuso.
- **Consistencia:** mismo estilo de nombres, formato y organización en todo el proyecto, lo que facilita leer y navegar el código.
- **Evitar duplicidad (DRY) y complejidad innecesaria (KISS, YAGNI):** no repetir lógica, no anticipar funcionalidades que "quizá" se necesiten en el futuro.

Ejemplo: una función `calcular_total_pedido()` con buen nombre, pocas líneas y sin efectos secundarios inesperados es más "limpia" que una función genérica `procesar()` que hace muchas cosas distintas.

1.2. Principios SOLID

Los principios SOLID son cinco reglas para diseñar clases y módulos orientados a objetos de manera flexible y mantenible.

Sigla	Principio	Descripción simple
S	Single Responsibility Principle (SRP)	Cada clase o módulo debe tener una sola razón para cambiar, es decir, una única responsabilidad clara

O	Open/Closed Principle (OCP)	Las clases deben estar abiertas a extensión, pero cerradas a modificación; se amplía el comportamiento añadiendo código nuevo en lugar de editar el existente.
L	Liskov Substitution Principle (LSP)	Las subclases deben poder reemplazar a sus clases base sin romper el comportamiento esperado del sistema.
I	Interface Segregation Principle (ISP)	Es mejor tener interfaces pequeñas y específicas que una interfaz “gorda” que obliga a implementar métodos que no se usan.
D	Dependency Inversion Principle (DIP)	Los módulos de alto nivel no deben depender de detalles concretos, sino de abstracciones (interfaces), facilitando pruebas e intercambiar implementaciones.

Tabla 1 - Principios SOLID - Fuente: Autor de Contenidos

La calidad del software se basa en principios fundamentales.

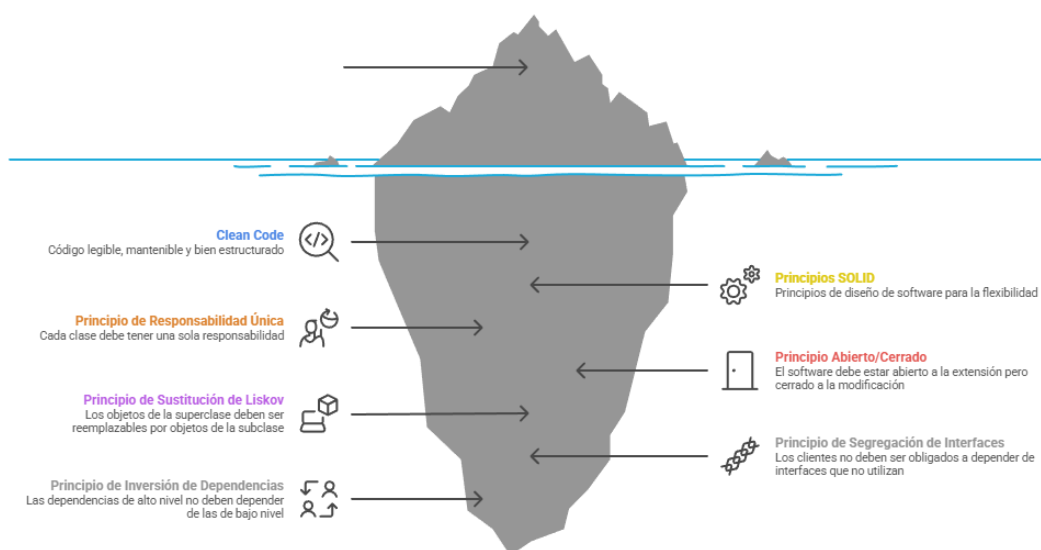


Ilustración 1 - Calidad de software- Fuente: Autor de contenidos

Clean Code y los principios SOLID se conectan como dos caras de la misma moneda en la búsqueda de software mantenible: Clean Code establece las prácticas cotidianas para escribir código legible, simple y modular —con nombres claros, funciones pequeñas y sin duplicación—, mientras que SOLID eleva esto a un nivel arquitectónico mediante cinco reglas clave (Responsabilidad Única, Abierto/Cerrado, Sustitución de Liskov, Segregación de Interfaces y Inversión de Dependencias) que aseguran diseños orientados a objetos flexibles, escalables y de bajo acoplamiento; juntos, facilitan la colaboración en equipos, reducen bugs a largo plazo y hacen que refactorizar sea intuitivo, convirtiendo código "sucio" en una base sólida para proyectos como APIs o aplicaciones web.

2. Marco de trabajo Scrum

Scrum es un marco de trabajo ágil utilizado para gestionar proyectos complejos mediante ciclos iterativos e incrementales llamados Sprints (generalmente de 1 a 4 semanas). Se basa en equipos autoorganizados y multidisciplinarios que entregan valor de forma eficiente, enfocándose en la transparencia, inspección y adaptación. La estructura se basa en 3 roles, 5 eventos y 3 artefactos (3-5-3).

Los Tres Pilares de Scrum

Componentes Clave de Scrum

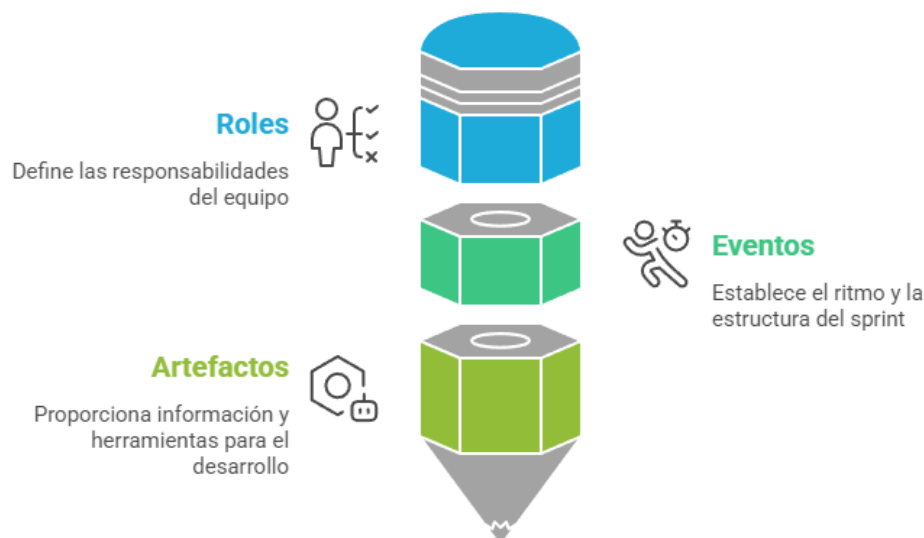


Ilustración 2 – Componentes claves de Scrum - Fuente: Autor de contenidos

Roles clave

- **Product Owner:** Representa al cliente y stakeholders; define prioridades en el Product Backlog, maximiza el valor del producto y acepta entregas.

- **Scrum Master:** Facilita el proceso, elimina obstáculos, asegura el cumplimiento de Scrum y fomenta la mejora continua del equipo.
- **Equipo de Desarrollo:** Grupo multidisciplinario (3-9 personas) autosuficiente que construye el incremento del producto de forma colaborativa

Eventos principales de Scrum:

- **Sprint:** Ciclo fijo (1-4 semanas) donde se crea un incremento potencialmente entregable; base de todo el marco.
- **Sprint Planning:** Planifica qué se hará en el Sprint y cómo; dura hasta 8 horas para un mes.
- **Daily Scrum:** Reunión diaria de 15 minutos para sincronizar avances, obstáculos y planes.
- **Sprint Review:** Inspecciona el incremento, adapta el Product Backlog y recopila feedback (hasta 4 horas).
- **Sprint Retrospective:** Reflexiona sobre el proceso para mejorar en el siguiente Sprint (hasta 3 horas).

Artefactos Esenciales

- **Product Backlog:** Lista priorizada y dinámica de todo lo necesario (features, bugs, etc.), propiedad del Product Owner.
- **Sprint Backlog:** Elementos seleccionados del Product Backlog más plan para completarlos; visible y detallado por el equipo.
- **Incremento:** Suma de todos los elementos completados, "Done" según definición de terminado; debe ser usable.

3. Flujos de Trabajo Eficientes con Kanban

Kanban optimiza flujos de trabajo visualizando tareas en tableros, limitando el trabajo en progreso y fomentando mejoras continuas para entregar valor de forma constante sin interrupciones bruscas.



Ilustración 3 – Principios fundamentales de Kaban - Fuente: Autor de contenidos

Tablero Kanban Típico

Columna	Propósito	Limite WIP Ejemplo
Backlog/To Do	Tareas pendientes priorizadas	Sin límite
En Progreso	Trabajo activo	3-5 tareas
En Revisión	Esperando QA o aprobación	2 tareas
Hecho	Completado y listo para entrega	Sin límite

Tabla 2 - Estructura tablero Kaban - Fuente: Autor de Contenidos

Prácticas para Flujos Eficientes

- Sistema Pull: Tareas se "tiran" solo cuando hay capacidad, reduciendo desperdicios y alineando con demanda real.

- Métricas clave: Tiempo de ciclo (de inicio a fin), lead time y throughput para iterar mejoras.
- Integración con herramientas: Usa Trello, Jira o tableros físicos para desarrollo backend, gestión de reservas o creación de cursos.

Kanban complementa Scrum en tu flujo (ej. para mantenimiento post-Sprint) y se adapta a equipos de 4-10 personas.

Scrum y Kanban son enfoques ágiles diferentes, pero no opuestos. Scrum se basa en un marco de trabajo estructurado con roles definidos, eventos formales y trabajo organizado en iteraciones llamadas sprints, lo que permite planificar y entregar valor de manera incremental y predecible. Por su parte, Kanban se enfoca en la visualización del flujo de trabajo y en la mejora continua mediante la gestión del trabajo en progreso, sin imponer roles ni ciclos de tiempo fijos.

Aunque sus enfoques son distintos, ambos pueden complementarse eficazmente. Scrum aporta estructura, planificación y objetivos claros, mientras que Kanban añade flexibilidad, control del flujo y optimización de la carga de trabajo. Al combinarse, los equipos pueden mantener la disciplina de Scrum y, al mismo tiempo, beneficiarse de la adaptabilidad y eficiencia que ofrece Kanban, logrando así una gestión del trabajo más equilibrada y efectiva.

Recursos de Apoyo



Guia

[Guía Oficial de Scrum : PDF descargable con ejemplos de roles, eventos y artefactos; incluye ejercicios para simulaciones de Sprints.](#)



Video

[Tutorial Interactivo Clean Code en Python: Ejemplos reales de refactorización antes/después](#)



Video

[Plantilla Kanban Interactiva en Trello: Tablero editable con límites WIP preconfigurados, tutorial para personalizar flujos de desarrollo web.](#)